

Ramulator 2.1: A Composable Memory System Simulator for Modern DRAM Systems

Haocong Luo F. Nisa Bostancı Ataberk Olgun Maria Makeenkova
Ziad Malik Ipek Akdeniz Onur Mutlu

SAFARI Research Group
ETH Zürich

Ramulator 2.1 is a major overhaul of Ramulator 2.0 that substantially improves the simulator in three directions: 1) support of modern and emerging DRAM and memory-controller features, 2) better usability and extensibility of the simulator, and 3) more comprehensive tests and validation workflows. Ramulator 2.1 adds support for advanced features in recent and emerging DRAM standards and memory controllers, including HBM3/4, LPDDR5/6, and GDDR7. To improve usability and extensibility, Ramulator 2.1 introduces a Python-based modeling and configuration interface backed by a two-way code-generation framework that 1) hides low-level C++ code behind high-level DRAM specifications written in Python, and 2) automatically creates Python proxies for all components of the simulator. Doing so enables users to rapidly create variants of DRAM standards and automate design-space-exploration workflows. To improve trustworthiness in simulation results, Ramulator 2.1 provides a comprehensive testing and validation infrastructure that covers both 1) fine-grained validation of specific DRAM timing constraints and memory-controller scheduling behavior, and 2) system-level performance evaluation using latency-throughput curves. To aid performance analysis and debugging, Ramulator 2.1 also includes an easy-to-use and high-performance DRAM command trace visualizer. Ramulator 2.1 is open-source on GitHub and under active development.

1. Introduction

Ramulator 2.1 [1] is a major overhaul of Ramulator 2.0 [1–5] that substantially improves 1) the coverage of modern DRAM [6] standards and memory-controller mechanisms, 2) the usability and extensibility of the simulator, and 3) the confidence users can place in simulation results.

First, Ramulator 2.1 expands support for protocol-level features in recent and emerging DRAM standards and memory controllers that are absent in DDR4: *parallel row/column command issue* in HBM3/HBM4 [7, 8] and GDDR7 [9], *split two-phase (ACT-1/ACT-2) activation* in LPDDR5/LPDDR6 [10, 11], and *data-clock synchronization* (WCK in LPDDR5/LPDDR6, RCK in GDDR7) [9–11]. Ramulator 2.1 captures these features within a modular software framework that factors common mechanisms into a shared controller hierarchy and composable filtering predicates, letting each standard inject its protocol-specific scheduling constraints into a common command-selection pipeline while preserving standard-specific behavior. Such a design also enables future extensions of different DRAM request scheduling policies (e.g., [12–15]).

Second, Ramulator 2.1 introduces a Python-based DRAM

modeling and simulation configuration interface supported by a two-way code-generation framework. This framework automatically 1) translates high-level DRAM specifications authored in Python into low-level C++ implementations compatible with Ramulator’s cycle-level simulation methodology [1–4] and 2) generates Python proxies and exposes configurable parameters for all other C++ simulator components. This design enables users to leverage Python’s expressiveness and scriptability to 1) easily create variants of existing DRAM standards (e.g., adding a new DRAM command), and 2) streamline and automate experiment workflows for design space exploration and evaluation.

Third, Ramulator 2.1 strengthens the trustworthiness of the simulation results with a comprehensive testing and validation infrastructure. It supports both 1) fine-grained and low-level validation of specific DRAM timing constraints and memory-controller scheduling behavior, and 2) system-level performance evaluation using latency-throughput curves. To aid performance analysis and debugging, Ramulator 2.1 also includes an easy-to-use and high-performance web-based DRAM command trace visualizer.

Ramulator 2.1 is actively maintained, and we will continue incorporating new DRAM features and functionalities, including Processing-Using-DRAM (PUD) [16–43] support and DRAM-based Processing-In-Memory (PIM) [44–64], as the project evolves. We very much welcome feedback and contributions from the broader community, including both academia and industry. To this end, please use our GitHub repository [1], where we actively monitor and handle any raised issues.

2. Modern DRAM and Controller Features

Modern DRAM standards introduce key new features at the controller-device protocol level to enable high-performance and power-efficient operation. For example: 1) HBM3/4 [7, 8] and GDDR7 [9] allow *parallel row/column command issue*, 2) LPDDR5/6 [10, 11] have *split two-phase activation*, and 3) LPDDR5/6 [10, 11] and GDDR7 [9] introduce *data-clock synchronization* (WCK and RCK).

To implement these features in a clean and modular way, Ramulator 2.1 first implements a common abstract DRAM controller base class with a high-level memory request scheduling workflow. Second, Ramulator 2.1 extends this base class into different controller classes that implement different features with only small additions and/or modifications on top of the base workflow. To facilitate the clean and modular implemen-

tation, Ramulator 2.1 allows passing *filtering predicates* in the form of lambdas into the base workflow to let each controller class inject feature-specific scheduling constraints.

Parallel row/column command issue. HBM3, HBM4, and GDDR7 provide separate C/A (i.e., command/address) buses for row commands (e.g., ACT) and column commands (e.g., RD, WR), allowing simultaneous issue of row and column commands. Ramulator 2.1 implements this feature by calling the base scheduling workflow *twice*, each with a different filtering predicate that selects only row or column commands.

Split ACT-1/ACT-2 activation. LPDDR5 and LPDDR6 split activation into two commands: 1) ACT-1 that transitions a bank to an intermediate *Activating* state, and 2) ACT-2 which must follow ACT-1 within the deadline of t_{AAD} cycles to complete the activation. To enable correct per-bank ACT-2 issuing and deadline tracking, Ramulator 2.1’s LPDDR5/6 controller passes filtering predicates that 1) only allow requests that have already issued ACT-1 to issue ACT-2, and 2) prevent other requests from interrupting the pending issue of ACT-2.

WCK/RCK data-clock synchronization. To save power, the faster data transfer clocks of LPDDR5/6 [10, 11] (WCK) and GDDR7 [9] (RCK) can be enabled only when there is data transfer. To do so, the controller must issue the corresponding synchronization commands (i.e., CAS_RD/CAS_WR for LPDDR5/6 and RCKSTRT/RCKSTOP for GDDR7) before the next read/write accesses. Ramulator 2.1 models this feature by explicitly tracking the data-clock active and expiry windows and injecting the required synchronization commands as prerequisites when needed, while preserving normal timing and scheduling constraints.

Other controller features. Filtering predicates also facilitate the implementation of other controller features that change DRAM request and scheduling behavior. For example, Ramulator 2.1 implements BlockHammer [65] and PRAC [66–68] controllers that use filtering predicates to 1) defer unsafe activation commands, and 2) ensure that ordinary requests do not interfere with required recovery or mitigation commands. We plan to implement many more DRAM request scheduling policies (e.g., [12–15, 69–75]) in the future.

3. Python Interface

Ramulator 2.1 provides a straightforward and easy-to-use Python interface that serves two purposes: *configuring simulations* and *authoring DRAM standards*. A code generation framework *automatically* 1) generates Python proxies and exposes configurable parameters and knobs for the actual Ramulator 2.1 code implemented in C++ so that a simulation can be flexibly configured and driven in Python, and 2) translates high-level and human readable DRAM specifications in Python into low-level C++ code for Ramulator’s cycle-level and hierarchical DRAM modeling and simulation methodology [1–4] that reduces repetitive and boilerplate code exposed to the user.

3.1. Simulation Configuration

Ramulator 2.1 comes with a Python package that consists of Python proxy classes for each of the simulator components (e.g., frontend, controller, scheduler, address mapper) in the C++ codebase. Each Python proxy class contains the same set of configurable parameters as the corresponding C++ components. With the Python package, users can flexibly instantiate and compose these proxy classes to configure the simulated system. Users can then run the simulation and collect structured statistics, all from a single Python script. The Ramulator 2.1 Python package can also be seamlessly integrated with other simulators with a Python-based configuration system like gem5 [76, 77]. The Python package and all proxy classes are *automatically generated* from the C++ source code during the build process, without the need for any manual maintenance (i.e., transparently to the user).

Under the hood, the Python proxies are lightweight and structured objects that only mirror the hierarchy and store the configurations of each simulator component *without* any binding to the actual C++ object. Doing so allows other simulators that do not use Python to configure the simulation to also integrate Ramulator 2.1 *without* needing to include extra Python dependencies. Ramulator 2.1 provides a tool to *automatically* generate an *equivalent* pure text YAML configuration file from the Python configuration script that can be directly parsed by the C++ codebase.

3.2. Authoring and Extending DRAM Standards

The Python package of Ramulator 2.1 also provides a high-level, human-readable, and easy-to-extend interface for authoring DRAM standards. The DRAM specifications (e.g., the DRAM organization hierarchy, supported DRAM commands, timing constraints) are described as plain Python dictionaries and lists of strings and numbers that directly mirror the names and values of DRAM organization, commands, and timing constraints, etc. The code generation framework then *automatically* generates the low-level C++ code following Ramulator’s cycle-level and hierarchical DRAM modeling and simulation methodology [1–4], hiding complicated, repetitive and boilerplate code from the user. For example, in Ramulator 2.1, the lines of code needed for DDR5 [66] reduce by 67.2% compared to Ramulator 2.0 [1, 4] (from 402 lines of C++ code down to 132 lines of Python code).

The Python-based DRAM specification authoring interface significantly simplifies the process of creating variations of DRAM standards by adding new DRAM commands and related timing constraints. Leveraging the flexibility and expressiveness of Python, to add a new DRAM command and timing constraints to a DRAM standard, users only need to 1) inherit from the base DRAM standard, and 2) append only the new commands and timings to the list of supported commands and timing constraints in the specification. For example, Listing 1 shows how to add a Victim Row Refresh [78] (VRR) command and its associated timing constraints on top of DDR5 with only 18 lines of Python source code (excluding blank and comment lines; compared to fully duplicating the entire 445 lines

Listing 1: Extending DDR5 with a new Victim Row Refresh (VRR) command and its associated timing constraints.

```

1 import math
2
3 from ramulator.dram.ddr5 import DDR5
4 from ramulator.dram.spec import TimingConstraint
5
6 # Inherit from DDR5
7 class DDR5_VRR(DDR5):
8     name = "DDR5_VRR"
9     # Append the new VRR command
10    commands = DDR5.commands + ["VRR"]
11    # Append the new timing constraints related to VRR
12    timing_params = DDR5.timing_params + ["nVRR"]
13    timing_constraints = DDR5.timing_constraints + [
14        TimingConstraint(level="Bank", preceding=["VRR"], following=["
15            ACT"], latency="nVRR"),
16        TimingConstraint(level="Bank", preceding=["ACT"], following=["
17            VRR"], latency="nRC"),
18        TimingConstraint(level="Rank", preceding=["PREpb", "PREab"],
19            following=["VRR"], latency="nRP"),
20    ]
21
22 # Reuse all DDR5 presets
23 DDR5_VRR.org_presets = DDR5.org_presets
24 DDR5_VRR.timing_presets = {}
25 # Add the new nVRR timing constraint to all DDR5 presets
26 for _name, _timings in DDR5.timing_presets.items():
27     _vrr_timings = dict(_timings)
28     _vrr_timings["nVRR"] = math.ceil(280_000 / _timings["tCK_ps"])
29     DDR5_VRR.timing_presets[_name] = _vrr_timings

```

Table 1: Reduction in lines of source code of DRAM standards in Ramulator 2.1 (Python) compared to Ramulator 2.0 (C++).

DRAM Standard	Lines of Source Code		Reduction
	v2.0 (C++)	v2.1 (Python)	
DDR3	325	129	60.3%
DDR4	354	161	54.5%
DDR5	402	132	67.2%
GDDR6	327	199	39.1%
HBM1	287	133	53.7%
HBM2	289	146	49.5%
LPDDR5	395	143	63.8%
DDR4-VRR	375	18	95.2%
DDR5-VRR	445	18	96.0%
Total	3,199	1,079	66.3%

of C++ code in Ramulator 2.0). Table 1 reports the reduction in lines of source code (excluding blank and comment lines) across the DRAM standards and command variants currently implemented in Ramulator 2.1.

4. Tests and Validation

To improve the trustworthiness of Ramulator 2.1’s modeling and simulation results, we implement a comprehensive testing and validation infrastructure that covers both 1) fine-grained and low-level validation of specific DRAM timing constraints and memory-controller scheduling behavior, and 2) system-level performance evaluation using latency-throughput curves.

Fine-Grained Validation. We implement a unit test framework based on pytest [79] that allows users to easily 1) create a device (DRAM or controller) under test, 2) send DRAM com-

Listing 2: An example fine-grained unit test for DDR4.

```

1 import pytest
2
3 import ramulator
4 import tests.device_timings.harness as device_timings
5
6 pytestmark = pytest.mark.device_timings
7
8 def dram_example_test():
9     # Create the DRAM under test
10    dram = ramulator.dram.DDR4(
11        org_preset="DDR4_8Gb_x8", timing_preset="DDR4_2400R", rank=1
12    )
13    dut = device_timings.DeviceUnderTest(dram)
14
15    addr = dut.addr_vec(Rank=0, BankGroup=0, Bank=0, Row=12, Column=0)
16
17    # Probe the states of the DRAM for a RD command at cycle 0
18    closed = dut.probe("RD", addr, clk=0)
19    # Check: The prerequisite command is ACT.
20    assert closed.preq == "ACT"
21    # Check: Timing is OK here since no ACT has been issued yet!
22    assert closed.timing_OK is True
23    # Check: Not ready since the prerequisite is not met.
24    assert closed.ready is False
25
26    # Issue the ACT command at cycle 0.
27    dut.issue("ACT", addr, clk=0)
28
29    # Probe and Check: Before nRCD, the row state is correct for RD
30    # but timing still blocks it.
31    early = dut.probe("RD", addr, clk=dut.timings["nRCD"] - 1)
32    assert early.preq == "RD"
33    assert early.timing_OK is False
34    assert early.ready is False
35    assert early.row_hit is True
36    assert early.row_open is True
37
38    # At nRCD, the same command becomes legal.
39    ontime = dut.probe("RD", addr, clk=dut.timings["nRCD"])
40    assert ontime.preq == "RD"
41    assert ontime.timing_OK is True
42    assert ontime.ready is True

```

mands or memory requests to the device, and 3) probe and check the internal states (e.g., prerequisite commands, timings) at specific cycles. Listing 2 shows an example unit test that verifies a read command to a DDR4 device is blocked until the row is activated and the nRCD timing constraint is satisfied. We aim to keep improving the test coverage and ease of use of this framework as we keep developing and maintaining Ramulator 2.1, and we welcome contributions from the community.

System-Level Performance Evaluation. We perform high-level evaluation of the memory system performance by measuring the DRAM access latency under different loads (memory system throughput). To do so, we implement an improved version of the traffic-generator frontend from earlier work [5] that generates two kinds of memory requests: 1) streaming requests with variable intervals between consecutive requests to adjust the load (throughput) of the memory system, and 2) serialized random-access requests that we use to probe the access latency (i.e., a probe request is issued only after the previous probe request completes). Figure 1 shows the latency-throughput curves for all DRAM standards modeled in Ramulator 2.1. The y-axis shows the average access latency measured from the random-access probe requests, and the x-axis shows the throughput of the memory system. Different colors correspond to different read ratios (i.e., the fraction of read requests

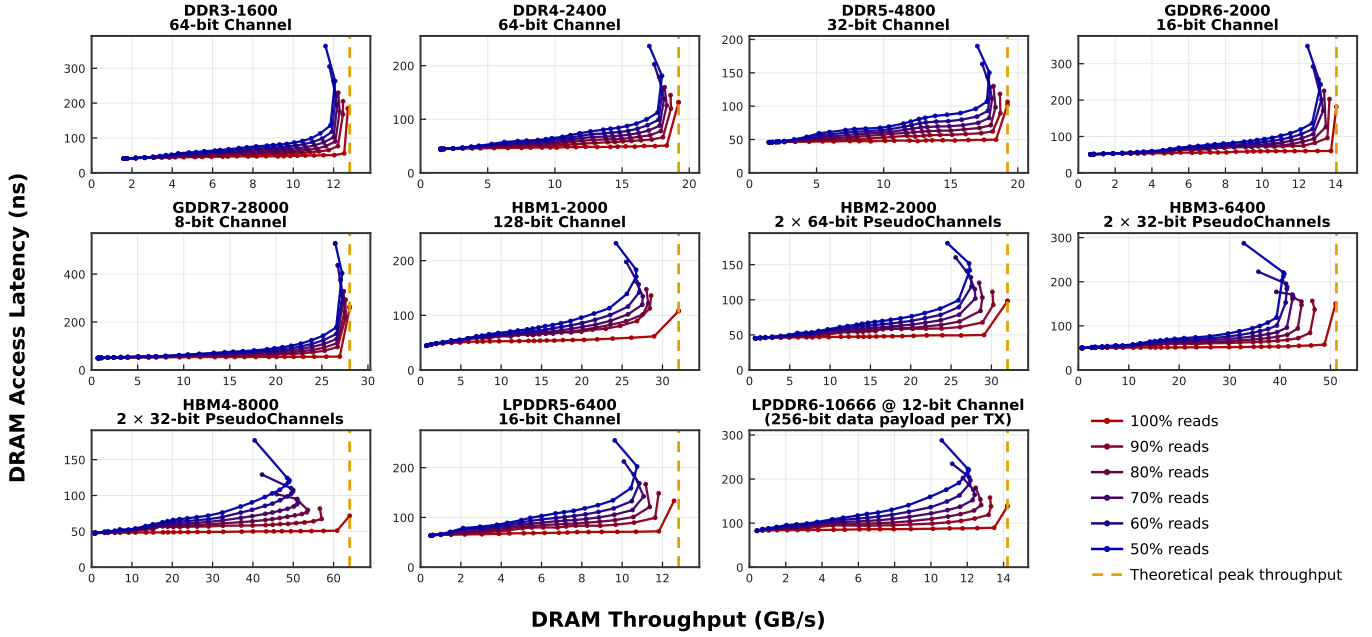


Figure 1: Latency-throughput curves across the DRAM standards modeled in Ramulator 2.1.

generated by the frontend, from 100% reads, red, down to 50% reads and 50% writes, blue). We mark the theoretical peak throughput of each DRAM standard as vertical dashed orange lines. We observe that 1) for all DRAM standards, the simulated memory system can achieve the theoretical peak throughput, and 2) the latency-throughput curves follow the expected knee-curve shape. We conclude that Ramulator 2.1’s modeling and simulation results are consistent with the expected behavior of real DRAM devices.

4.1. DRAM Command Trace Visualizer

To help users analyze DRAM system performance and debug the DRAM and memory controller models, Ramulator 2.1 includes a web-based, easy-to-use, and high-performance DRAM command trace visualizer. The visualizer can either 1) parse and visualize a recorded DRAM command trace offline, or 2) connect to a running Ramulator 2.1 simulation and visualize the DRAM commands being issued in real time. Users can easily inspect the DRAM command sequence, timings, addresses, and the address and data bus utilization interactively through the visualizer. Figure 2 shows two screenshots from the visualizer: (a) the bus utilization view illustrating the data bus and command bus utilization, and (b) the command trace view showing the sequence of DRAM commands and their addresses issued over time.

5. Conclusion

We introduce Ramulator 2.1, a major overhaul of Ramulator 2.0 that improves the simulator along three directions. First, it expands support for protocol-level features in recent and emerging DRAM standards and memory controllers. Second, it introduces a Python-based DRAM modeling and simulation configuration interface, enabling users to rapidly create variants of DRAM standards and automate design-space-exploration

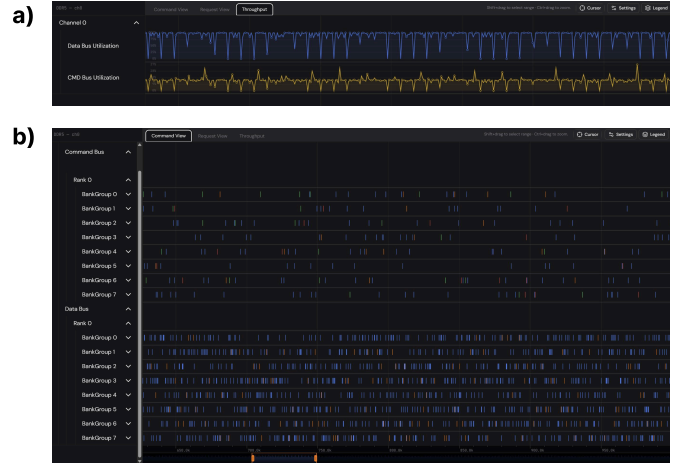


Figure 2: Ramulator 2.1’s DRAM command trace visualizer: (a) the bus utilization view and (b) the command trace view.

workflows. Third, it strengthens the trustworthiness of simulation results with a comprehensive testing and validation infrastructure that spans fine-grained DRAM-timing and scheduling checks and system-level latency-throughput evaluation. Our results show that Ramulator 2.1 significantly reduces the lines of code needed to implement DRAM standards compared to Ramulator 2.0. Ramulator 2.1 also provides latency-throughput curves that match curves expected from real systems for 11 DRAM standards.

Ramulator 2.1 is open-source and under active development. We will continue to incorporate new DRAM features and functionalities. We are very open to and warmly welcome feedback and contributions from both academia and industry through our GitHub repository [1] to improve Ramulator 2.1.

Acknowledgments

We thank the SAFARI Research Group members for their constructive feedback and for providing a stimulating intellectual, scholarly, and scientific environment. We acknowledge the generous gift funding provided by our industrial partners (especially Google, Huawei, Intel, Microsoft), which has been instrumental in enabling the research we have been conducting on read disturbance in DRAM in particular and memory systems in general [22,62,63,78,80–103]. This work was in part supported by a Google Security and Privacy Research Award and the Microsoft Swiss Joint Research Center.

References

- [1] SAFARI Research Group, “Ramulator 2 — GitHub Repository,” 2023. [Online]. Available: <https://github.com/CMU-SAFARI/ramulator2>
- [2] Y. Kim, W. Yang, and O. Mutlu, “Ramulator: A Fast and Extensible DRAM Simulator,” *CAL*, 2016.
- [3] SAFARI Research Group, “Ramulator — GitHub Repository,” 2016. [Online]. Available: <https://github.com/CMU-SAFARI/ramulator>
- [4] H. Luo, Y. C. Tugrul, F. N. Bostanci, A. Olgun, A. G. Yaglikci, and O. Mutlu, “Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator,” *CAL*, 2023.
- [5] F. N. Bostanci, H. Luo, A. Olgun, M. Makeenkova, G. F. Oliveira, A. G. Yaglikci, and O. Mutlu, “Cleaning up the Mess: Re-Evaluating the Real-System Modeling Accuracy of Ramulator 2.0,” in *ISPASS*, 2026.
- [6] R. H. Dennard, “Field-Effect Transistor Memory,” 1968, U.S. Patent 3,387,286.
- [7] JEDEC Solid State Technology Association, “High Bandwidth Memory (HBM3),” JEDEC, JEDEC Standard JESD238, 2022.
- [8] JEDEC Solid State Technology Association, “High Bandwidth Memory (HBM4) DRAM,” JEDEC, JEDEC Standard JESD270-4, 2025.
- [9] JEDEC Solid State Technology Association, “Graphics Double Data Rate (GDDR7) SGRAM,” JEDEC, JEDEC Standard JESD239, 2024.
- [10] JEDEC Solid State Technology Association, “Low Power Double Data Rate (LPDDR) 5/5X,” JEDEC, JEDEC Standard JESD209-5, 2019.
- [11] JEDEC Solid State Technology Association, “Low Power Double Data Rate (LPDDR) 6,” JEDEC, JEDEC Standard JESD209-6, 2025.
- [12] Y. Kim, M. Papamichael, O. Mutlu, and M. Harchol-Balter, “Thread Cluster Memory Scheduling: Exploiting Differences in Memory Access Behavior,” in *MICRO*, 2010.
- [13] O. Mutlu and T. Moscibroda, “Parallelism-Aware Batch Scheduling: Enhancing both Performance and Fairness of Shared DRAM Systems,” in *ISCA*, 2008.
- [14] Y. Kim, D. Han, O. Mutlu, and M. Harchol-Balter, “ATLAS: A Scalable and High-Performance Scheduling Algorithm for Multiple Memory Controllers,” in *HPCA*, 2010.
- [15] O. Mutlu and T. Moscibroda, “Stall-Time Fair Memory Access Scheduling for Chip Multiprocessors,” in *MICRO*, 2007.
- [16] V. Seshadri, K. Hsieh, A. Boroumand, D. Lee, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, “Fast Bulk Bitwise AND and OR in DRAM,” *CAL*, 2015.
- [17] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, “Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology,” in *MICRO*, 2017.
- [18] N. Hajimazari, G. F. Oliveira, S. Gregorio, J. D. Ferreira, N. M. Ghiasi, M. Patel, M. Alser, S. Ghose, J. Gómez-Luna, and O. Mutlu, “SIMDRAM: A Framework for Bit-Serial SIMD Processing Using DRAM,” in *ASPLOS*, 2021.
- [19] I. E. Yuksel, Y. C. Tugrul, F. N. Bostanci, G. F. de Oliveira, A. G. Yaglikci, A. Olgun, M. Soysal, H. Luo, J. G. Luna, M. Sadrosadati, and O. Mutlu, “Simultaneous Many-Row Activation in Off-the-Shelf DRAM Chips: Experimental Characterization and Analysis,” in *DSN*, 2024.
- [20] G. F. Oliveira, A. Olgun, A. G. Yaglikci, F. N. Bostanci, J. Gómez-Luna, S. Ghose, and O. Mutlu, “MIMDRAM: An End-to-End Processing-Using-DRAM System for High-Throughput, Energy-Efficient and Programmer-Transparent Multiple-Instruction Multiple-Data Processing,” in *HPCA*, 2024.
- [21] I. E. Yuksel, Y. C. Tugrul, A. Olgun, F. N. Bostanci, A. G. Yaglikci, G. F. de Oliveira, H. Luo, J. G. Luna, M. Sadrosadati, and O. Mutlu, “Functionally-Complete Boolean Logic in Real DRAM Chips: Experimental Characterization and Analysis,” in *HPCA*, 2024.
- [22] O. Mutlu, A. Olgun, G. F. Oliveira, and I. E. Yuksel, “Memory-Centric Computing: Recent Advances in Processing-in-DRAM,” in *IEDM*, 2024.
- [23] A. Olgun, J. G. Luna, K. Kanellopoulos, B. Salami, H. Hassan, O. Ergin, and O. Mutlu, “PiDRAM: A Holistic End-to-end FPGA-based Framework for Processing-in-DRAM,” *TACO*, 2022.
- [24] G. F. de Oliveira Junior, M. Kabra, Y. Guo, K. Chen, A. G. Yaglikci, M. Soysal, M. Sadrosadati, J. Olivares Bueno, S. Ghose, J. Gómez-Luna, and O. Mutlu, “Proteus: Achieving High-Performance Processing-Using-DRAM with Dynamic Bit-Precision, Adaptive Data Representation, and Flexible Arithmetic,” in *ICS*, 2025.
- [25] J. S. Kim, M. Patel, H. Hassan, and O. Mutlu, “The DRAM Latency PUF: Quickly Evaluating Physical Unclonable Functions by Exploiting the Latency–Reliability Tradeoff in Modern Commodity DRAM Devices,” in *HPCA*, 2018.
- [26] J. S. Kim, M. Patel, H. Hassan, L. Orosa, and O. Mutlu, “D-RaNGe: Using Commodity DRAM Devices to Generate True Random Numbers with Low Latency and High Throughput,” in *HPCA*, 2019.
- [27] A. Olgun, M. Patel, A. G. Yaglikci, H. Luo, J. S. Kim, N. Bostanci, N. Vijaykumar, O. Ergin, and O. Mutlu, “QUAC-TRNG: High-Throughput True Random Number Generation Using Quadruple Row Activation in Commodity DRAM Chips,” in *ISCA*, 2021.
- [28] F. Gao, G. Tziantzioulis, and D. Wentzlaff, “ComputeDRAM: In-Memory Compute Using Off-the-Shelf DRAMs,” in *MICRO*, 2019.
- [29] V. Seshadri, Y. Kim, C. Fallin, D. Lee, R. Ausavarungnirun, G. Pekhimenko, Y. Luo, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. Mowry, “RowClone: Fast and Energy-Efficient In-DRAM Bulk Data Copy and Initialization,” in *MICRO*, 2013.
- [30] U. Baser, I. E. Yuksel, F. N. Bostanci, K. Sgouras, A. Olgun, E. H. Demirli, Z. Yue, H. Songara, O. Ergin, and O. Mutlu, “In-DRAM Signature Generation Using Simultaneous Multiple-Row Activation: An Experimental Study of Off-The-Shelf DRAM Chips,” *DSN Disrupt*, 2026.
- [31] I. E. Yuksel, A. Olgun, N. Bostanci, O. Canpolat, G. F. Oliveira, M. Sadrosadati, A. G. Yaglikci, and O. Mutlu, “In-DRAM True Random Number Generation Using Simultaneous Multiple-Row Activation: An Experimental Study of Real DRAM Chips,” in *ICCD*, 2025.
- [32] D. Tokuda, T. Kubo, I. E. Yuksel, A. Olgun, H. Luo, T. Nagatani, G. F. Oliveira, A. G. Yaglikci, M. Sadrosadati, O. Mutlu, and S. Takamaeda-Yamazaki, “Clutch: High Performance Vector-Scalar Comparison using DRAM via Chunked Temporal Coding,” in *ICS*, 2026.
- [33] D. Tokuda, I. E. Yuksel, T. Kubo, A. Olgun, H. Luo, N. Bostanci, J. Wang, A. G. Yaglikci, S. Takamaeda-Yamazaki, and O. Mutlu, “PuDGHosh: Experimental Analysis of Computation Result Corruption in Processing-using-DRAM Operations on Real DRAM Chips and Implications for Future Systems,” in *ISCA*, 2026.
- [34] T. Kubo, M. Usui, T. Nagatani, D. Tokuda, L. Qu, T. Cao, and S. Takamaeda-Yamazaki, “Bulk Bitwise Accumulation in Commercial DRAM,” in *MLNCP*, 2024.
- [35] V. Seshadri and O. Mutlu, “In-DRAM Bulk Bitwise Execution Engine,” 2020. [Online]. Available: <https://arxiv.org/abs/1905.09822>
- [36] Q. Deng, L. Jiang, Y. Zhang, M. Zhang, and J. Yang, “DrAcc: A DRAM Based Accelerator for Accurate CNN Inference,” in *DAC*, 2018.
- [37] X. Xin, Y. Zhang, and J. Yang, “ELP2IM: Efficient and Low Power Bitwise Operation Processing in DRAM,” in *HPCA*, 2020.
- [38] H. Shin, R. Park, and J. W. Lee, “A Processing-using-Memory Architecture for Commodity DRAM Devices with Enhanced Compatibility and Reliability,” in *ICCAD*, 2024.
- [39] R. Wong, B. Feinberg, and S. Ghose, “DARTH-PUM: A Hybrid Processing-Using-Memory Architecture,” in *ASPLOS*, 2026.
- [40] M. S. Q. Truong, E. Chen, D. Su, L. Shen, A. Glass, L. R. Carley, J. A. Bain, and S. Ghose, “RACER: Bit-Pipelined Processing Using Resistive Memory,” in *MICRO*, 2021.
- [41] M. S. Q. Truong, Y. Sun, D. Xiong, A. Shah, A. Glass, A. Farrell, J. A. Bain, L. R. Carley, and S. Ghose, “The Memory Processing Unit: A Generalized Interface for End-to-End In-Memory Execution,” in *HPCA*, 2026.
- [42] F. Gao, G. Tziantzioulis, and D. Wentzlaff, “FracDRAM: Fractional Values in Off-the-Shelf DRAM,” in *MICRO*, 2022.
- [43] I. E. Yuksel, A. Sood, A. Olgun, O. Canpolat, H. Luo, N. Bostanci, M. Sadrosadati, A. G. Yaglikci, and O. Mutlu, “PuDHammer: Experimental Analysis of Read Disturbance Effects of Processing-using-DRAM in Real DRAM Chips,” in *ISCA*, 2025.
- [44] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, “Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product,” in *ISCA*, 2021.
- [45] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. N. Vijaykumar, “Newton: A DRAM-maker’s Accelerator-in-Memory (AiM) Architecture for Machine Learning,” in *MICRO*, 2020.
- [46] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An, J. Kim, J. Lee, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Kim, D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, M. Lee, M. Shin, M. Shin, J. Cha, C. Jung, K. Chang, C. Jeong, E. Lim, I. Park, and J. Chun, “System Architecture and Software Stack for GDDR6-AiM,” in *Hot Chips*, 2022.
- [47] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, K. Vladimir, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, J. Lee, D. Ko, Y. Jun, K. Cho, I. Kim, C. Song, C. Jeong, D. Kwon, J. Jang, I. Park, J. Chun, and J. Cho, “A 1ynm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications,” in *ISSCC*, 2022.
- [48] A. Farmahini-Farahani, J. H. Ahn, K. Morrow, and N. S. Kim, “NDA: Near-DRAM Acceleration Architecture Leveraging Commodity DRAM Devices and Standard Memory Modules,” in *HPCA*, 2015.
- [49] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, “A Scalable Processing-in-Memory Accelerator for Parallel Graph Processing,” in *ISCA*, 2015.
- [50] J. Ahn, S. Yoo, O. Mutlu, and K. Choi, “PIM-Enabled Instructions: A Low-Overhead, Locality-Aware Processing-in-Memory Architecture,” in *ISCA*, 2015.
- [51] L. Nai, R. Hadidi, J. Sim, H. Kim, P. Kumar, and H. Kim, “GraphPIM: Enabling Instruction-Level PIM Offloading in Graph Computing Frameworks,” in *HPCA*, 2017.
- [52] H. Asghari-Moghaddam, Y. H. Son, J. H. Ahn, and N. S. Kim, “Chameleon: Versatile and Practical Near-DRAM Acceleration Architecture for Large Memory Systems,” in *MICRO*, 2016.

- [53] E. Azarkhish, D. Rossi, I. Loi, and L. Benini, "Design and Evaluation of a Processing-in-Memory Architecture for the Smart Memory Cube," in *ARCS*, 2016.
- [54] A. Boroumand, S. Ghose, Y. Kim, R. Ausavarungnirun, E. Shiu, R. Thakur, D. Kim, A. Kuusela, A. Knies, P. Ranganathan, and O. Mutlu, "Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks," in *ASPLOS*, 2018.
- [55] H. Shin, D. Kim, E. Park, S. Park, Y. Park, and S. Yoo, "McDRAM: Low Latency and Energy-Efficient Matrix Computations in DRAM," *TCAD*, 2018.
- [56] D. Zhang, N. Jayasena, A. Lyashevsky, J. L. Greathouse, L. Xu, and M. Ignatowski, "TOP-PIM: Throughput-Oriented Programmable Processing in Memory," in *HPDC*, 2014.
- [57] A. Boroumand, S. Ghose, B. Akin, R. Narayanaswami, G. F. Oliveira, X. Ma, E. Shiu, and O. Mutlu, "Google Neural Network Models for Edge Devices: Analyzing and Mitigating Machine Learning Inference Bottlenecks," in *PACT*, 2021.
- [58] Y. He, H. Mao, C. Giannoula, M. Sadrosadati, J. Gómez-Luna, H. Li, X. Li, Y. Wang, and O. Mutlu, "PAPI: Exploiting Dynamic Parallelism in Large Language Model Decoding with a Processing-In-Memory-Enabled Computing System," in *ASPLOS*, 2025.
- [59] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, "PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference," in *ASPLOS*, 2025.
- [60] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference," in *ASPLOS*, 2024.
- [61] S. Cho, H. Choi, E. Park, H. Shin, and S. Yoo, "McDRAM v2: In-Dynamic Random Access Memory Systolic Array Accelerator to Address the Large Model Problem in Deep Neural Networks on the Edge," *IEEE Access*, 2020.
- [62] G. F. Oliveira, J. Gómez-Luna, L. Orosa, S. Ghose, N. Vijaykumar, I. Fernandez, M. Sadrosadati, and O. Mutlu, "DAMOV: A New Methodology and Benchmark Suite for Evaluating Data Movement Bottlenecks," *IEEE Access*, 2021.
- [63] J. Gómez-Luna, I. E. Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System," *IEEE Access*, 2022.
- [64] F. Devaux, "The True Processing In Memory Accelerator," in *Hot Chips*, 2019.
- [65] A. G. Yağlıkcı, M. Patel, J. S. Kim, R. Azizbarzoki, A. Olgun, L. Orosa, H. Hassan, J. Park, K. Kanellopoulos, T. Shahroodi, S. Ghose, and O. Mutlu, "BlockHammer: Preventing RowHammer at Low Cost by Blacklisting Rapidly-Accessed DRAM Rows," in *HPCA*, 2021.
- [66] JEDEC Solid State Technology Association, "DDR5 SDRAM Standard," JEDEC, JEDEC Standard JESD79-5C, 2024.
- [67] O. Canpolat, A. G. Yağlıkcı, G. F. Oliveira, A. Olgun, N. Bostanci, I. E. Yuksel, H. Luo, O. Ergin, and O. Mutlu, "Chronus: Understanding and Securing the Cutting-Edge Industry Solutions to DRAM Read Disturbance," in *HPCA*, 2025.
- [68] O. Canpolat, A. G. Yağlıkcı, G. F. Oliveira, A. Olgun, O. Ergin, and O. Mutlu, "Understanding the Security Benefits and Overheads of Emerging Industry Solutions to DRAM Read Disturbance," in *DRAMSec*, 2024.
- [69] L. Subramanian, D. Lee, V. Seshadri, H. Rastogi, and O. Mutlu, "BLISS: Balancing Performance, Fairness and Complexity in Memory Access Scheduling," *TPDS*, 2016.
- [70] L. Subramanian, D. Lee, V. Seshadri, H. Rastogi, and O. Mutlu, "The Blacklisting Memory Scheduler: Achieving High Performance and Fairness at Low Cost," in *ICCD*, 2014.
- [71] H. Usui, L. Subramanian, K. K.-W. Chang, and O. Mutlu, "DASH: Deadline-Aware High-Performance Memory Scheduler for Heterogeneous Systems with Hardware Accelerators," *TACO*, 2016.
- [72] R. Ausavarungnirun, K. K.-W. Chang, L. Subramanian, G. H. Loh, and O. Mutlu, "Staged Memory Scheduling: Achieving High Performance and Scalability in Heterogeneous Systems," in *ISCA*, 2012.
- [73] Y. Kim, V. Seshadri, D. Lee, J. Liu, and O. Mutlu, "A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM," in *ISCA*, 2012.
- [74] A. G. Yağlıkcı, A. Olgun, M. Patel, H. Luo, H. Hassan, L. Orosa, O. Ergin, and O. Mutlu, "HIRA: Hidden Row Activation for Reducing Refresh Latency of Off-the-Shelf DRAM Chips," in *MICRO*, 2022.
- [75] K. K.-W. Chang, D. Lee, Z. Chishti, A. R. Alameldeen, C. Wilkerson, Y. Kim, and O. Mutlu, "Improving DRAM Performance by Parallelizing Refreshes with Accesses," in *HPCA*, 2014.
- [76] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoabi, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, 2011.
- [77] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, B. Beckmann, S. Bharadwaj, G. Black, G. Bloom, B. R. Bruce, D. R. Carvalho, J. Castrillon, L. Chen, N. Derumigny, S. Diestelhorst, W. Elsasser, C. Escuin, M. Fariborz, A. Farmahini-Farahani, P. Fotouhi, R. Gambord, J. Gandhi, D. Gope, T. Grass, A. Gutierrez, B. Hanindhito, A. Hansson, S. Haria, A. Harris, T. Hayes, A. Herrera, M. Horsnell, S. A. R. Jafri, R. Jagtap, H. Jang, R. Jayapaul, T. M. Jones, M. Jung, S. Kannoth, H. Khaleghzadeh, Y. Kodama, T. Krishna, T. Marinelli, C. Menard, A. Mondelli, M. Moreto, T. Mück, O. Naji, K. Nathella, H. Nguyen, N. Nikoleris, L. E. Olson, M. Orr, B. Pham, P. Prieto, T. Reddy, A. Roelke, M. Samani, A. Sandberg, J. Setoain, B. Shingarov, M. D. Sinclair, T. Ta, R. Thakur, G. Travaglini, M. Upton, N. Vaish, I. Vougioukas, W. Wang, Z. Wang, N. Wehn, C. Weis, D. A. Wood, H. Yoon, and Éder F. Zulian, "The gem5 Simulator: Version 20.0+," 2020. [Online]. Available: <https://arxiv.org/abs/2007.03152>
- [78] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, "Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors," in *ISCA*, 2014.
- [79] H. Krekel, B. Oliveira, R. Pfannschmidt, F. Bruynooghe, B. Laughner, and F. Bruhin, "pytest 9.0.2," 2025. [Online]. Available: <https://github.com/pytest-dev/pytest>
- [80] O. Mutlu, "The RowHammer Problem and Other Issues We May Face as Memory Becomes Denser," in *DATE*, 2017.
- [81] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "Processing Data Where It Makes Sense: Enabling In-Memory Computation," *Microprocessors and Microsystems*, 2019.
- [82] O. Mutlu and J. S. Kim, "RowHammer: A Retrospective," *TCAD*, 2019.
- [83] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "A Modern Primer on Processing in Memory," in *Emerging computing: from devices to systems: looking beyond Moore and Von Neumann*. Springer, 2022.
- [84] O. Mutlu, A. Olgun, and A. G. Yağlıkcı, "Fundamentally Understanding and Solving RowHammer," in *ASP-DAC*, 2023.
- [85] O. Mutlu, "Retrospective: Flipping Bits in Memory without Accessing Them: An Experimental Study of DRAM Disturbance Errors," *arXiv:2306.16093*, 2023.
- [86] O. Mutlu and L. Subramanian, "Research Problems and Opportunities in Memory Systems," *SUPERFRI*, 2014.
- [87] O. Mutlu, "Retrospective: RAIDR: Retention-Aware Intelligent DRAM Refresh," *arXiv:2306.16024*, 2023.
- [88] O. Mutlu, "Retrospective: An Experimental Study of Data Retention Behavior in Modern DRAM Devices: Implications for Retention Time Profiling Mechanisms," *arXiv:2306.16037*, 2023.
- [89] O. Mutlu, A. Olgun, and I. E. Yuksel, "Memory-Centric Computing: Solving Computing's Memory Problem," in *IMW*, 2025.
- [90] O. Mutlu, "Memory Scaling: A Systems Architecture Perspective," in *IMW*, 2013.
- [91] A. K. Kakolyris, F. N. Bostanci, A. Olgun, I. E. Yuksel, H. Songara, K. M. Sgouras, U. Baser, K. Kanellopoulos, A. G. Yağlıkcı, and O. Mutlu, "ColumnKeeper: Efficient Solutions for Mitigating ColumnDisturb in DRAM-based Systems," in *ISCA*, 2026.
- [92] S. Ghose, A. Boroumand, J. S. Kim, J. Gómez-Luna, and O. Mutlu, "Processing-in-Memory: A Workload-Driven Perspective," *IBM Journal of Research and Development*, 2019.
- [93] O. Mutlu, "Main Memory Scaling: Challenges and Solution Directions," in *More Than Moore Technologies for Next Generation Computer Design*. Springer, 2015.
- [94] G. F. Oliveira, J. Gómez-Luna, S. Ghose, A. Boroumand, and O. Mutlu, "Accelerating Neural Network Inference With Processing-in-DRAM: From the Edge to the Cloud," *IEEE Micro*, 2022.
- [95] G. Singh, M. Alser, D. S. Cali, D. Diamantopoulos, J. Gómez-Luna, H. Corporaal, and O. Mutlu, "FPGA-Based Near-Memory Acceleration of Modern Data-Intensive Applications," *IEEE Micro*, 2021.
- [96] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, "Error Characterization, Mitigation, and Recovery in Flash Memory Based Solid-State Drives," *Proc. IEEE*, 2017.
- [97] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, "Errors in Flash-Memory-Based Solid-State Drives: Analysis, Mitigation, and Recovery," *arXiv:1711.11427*, 2017.
- [98] O. Mutlu, "Intelligent Architectures for Intelligent Computing Systems," in *DATE*, 2020.
- [99] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "Retrospective: A Scalable Processing-in-Memory Accelerator for Parallel Graph Processing," *Retrospective Issue for ISCA-50*, 2023.
- [100] J. F. Martinez, E. Ipek, O. Mutlu, and R. Caruana, "Retrospective: Self-optimizing Memory Controllers: A Reinforcement Learning Approach," *Retrospective Issue for ISCA-50*, 2023.
- [101] I. E. Yuksel, F. N. Bostanci, A. Olgun, and O. Mutlu, "Memory-Centric Computing: Security Benefits and Challenges of Processing-in-DRAM," The 7th Workshop on Memory-Centric Computing Systems colocated with ICS, 2026.
- [102] A. Olgun, H. Luo, I. E. Yuksel, F. N. Bostanci, A. G. Yağlıkcı, and O. Mutlu, "A Modern Large-Scale Memory Characterization Laboratory," The 3rd Tutorial on Ramulator and DRAM Bender colocated with ICS, 2026.
- [103] F. N. Bostanci, H. Luo, A. Olgun, M. Makeenkova, G. F. de Oliveira, A. G. Yağlıkcı, and O. Mutlu, "Extended Abstract: Re-Evaluating the Real-System Modeling Accuracy of Ramulator 2.0," The 3rd Tutorial on Ramulator and DRAM Bender colocated with ICS, 2026.